**Theory Question**

---

**File Handling interview question**

---

Basic Questions

What is file handling in Java?
What are the different classes used for file handling?
What is the purpose of the File class?
Difference between FileWriter and FileOutputStream?
What is the difference between byte stream and character stream?

Concept-Based Questions

What is the difference between FileInputStream and BufferedInputStream?
What are Reader and Writer classes?
What is buffering and why is it used?
What is the difference between BufferedReader and Scanner?
What is PrintWriter used for?

Exception Handling Questions

What exceptions are commonly thrown in file handling?
What is IOException?
Difference between checked and unchecked exceptions in file handling?
Why must we handle or declare IOException?
What is try-with-resources?

Advanced Questions

What is serialization and deserialization?
What are ObjectInputStream and ObjectOutputStream?
What is the difference between Serializable and Externalizable?
What is a random access file in Java?
What is the difference between File and Path (NIO)?

Scenario-Based Questions

How do you read a large file efficiently?
How do you write data to a file without overwriting existing content?
How can you check if a file is readable or writable?
How do you copy content from one file to another in Java?
How would you handle file operations in a multi-threaded environment?

**JDK 1.8 version**

---

**Java 8 Interview Questions (Without Answers)**

**Core Features**

1. What are the major features introduced in Java 8?

2. What is a **lambda expression**?
3. What are functional interfaces?
4. What is the @FunctionalInterface annotation?
5. Difference between anonymous class and lambda expression?

---

**Lambda Expressions**

6. What is the syntax of a lambda expression?
7. Can lambda expressions access local variables?
8. What is “effectively final” variable in lambda?
9. Can we use this keyword inside lambda?
10. What are the limitations of lambda expressions?

---

**Functional Interfaces**

11. What is a functional interface?
12. Name some built-in functional interfaces.
13. Difference between Predicate, Function, and Consumer?
14. What is the Supplier interface?
15. Can a functional interface have multiple default methods?

---

**Stream API**

16. What is Stream API?
17. Difference between collection and stream?
18. What are intermediate and terminal operations?
19. What is lazy evaluation in streams?
20. What is map() vs flatMap()?

---

**Stream Operations**

21. What does filter() do?
22. What is forEach()?
23. What is collect()?
24. Difference between findFirst() and findAny()?
25. What is the reduce() method?

---

**Optional Class**

26. What is Optional?
27. Why was Optional introduced?
28. Difference between orElse() and orElseGet()?
29. What is orElseThrow()?
30. How to avoid NullPointerException using Optional?

---

**Default & Static Methods**

31. What are default methods in interfaces?
32. Why were default methods introduced?
33. Can interfaces have static methods?
34. What happens if two interfaces have the same default method?
35. Can default methods be overridden?

---

**Method References**

36. What is method reference?
37. Types of method references in Java 8?
38. Difference between lambda and method reference?
39. Syntax of method reference?
40. When should you prefer method reference over lambda?

---

**Date & Time API**

41. What is the new Date and Time API in Java 8?
42. Difference between old Date API and new API?
43. What is LocalDate, LocalTime, LocalDateTime?
44. What is ZonedDateTime?
45. How to format date in Java 8?

MCQ Question

---

**1. What will be the output?**

```
File file = new File("abc.txt");
System.out.println(file.exists());
```

A. true
B. false
C. Depends on file presence
D. Error

**Answer:** C
**Explanation:** Output depends on whether abc.txt exists.

---

**2.**

```
FileWriter fw = new FileWriter("a.txt");
fw.write(65);
fw.close();
```

A. 65
B. A
C. Error
D. Blank

**Answer:** B
**Explanation:** 65 is ASCII for 'A'.

---

**3.**

```
FileOutputStream fos = new FileOutputStream("a.txt");
fos.write(66);
fos.close();
```

A. 66
B. B
C. Error
D. Null

**Answer:** B
**Explanation:** Writes ASCII value → 'B'.

---

**4.**

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));
System.out.println(br.read());
```

A. First character
B. ASCII value
C. Error
D. Whole file

**Answer:** B
**Explanation:** read() returns ASCII (int).

---

**5.**

```
BufferedReader br = new BufferedReader(new FileReader("a.txt"));
System.out.println((char)br.read());
```

A. ASCII number
B. First character
C. Error
D. Null

**Answer:** B

---

**6.**

```
FileWriter fw = new FileWriter("a.txt", true);
fw.write("Hi");
fw.close();
```

A. Overwrites
B. Appends
C. Deletes
D. Error

**Answer:** B

---

**7.**

```
PrintWriter pw = new PrintWriter("a.txt");
pw.println("Hello");
pw.close();
```

A. Hello
B. Hello\n
C. Error
D. Blank

**Answer:** B

---

**8.**

```
Scanner sc = new Scanner(new File("a.txt"));
System.out.println(sc.next());
```

A. First word
B. First line
C. Whole file
D. Error

**Answer:** A

---

**9.**

```
File file = new File("a.txt");
System.out.println(file.canRead());
```

A. true/false
B. Always true
C. Error
D. Null
**Answer:** A

---

**10.**

```
File file = new File("a.txt");
System.out.println(file.length());
```

A. Size in bytes
B. Characters
C. Error
D. Lines
**Answer:** A

---

**11.**

```
try(FileWriter fw = new FileWriter("a.txt")){
   fw.write("Hi");
}
```

A. Error
B. Auto close works
C. No write
D. Runtime error
**Answer:** B

---

**12.**

```
ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("a.txt"));
oos.writeObject("Hello");
oos.close();
```

A. Error
B. Writes object
C. Writes plain text
D. Nothing
**Answer:** B

---

**13.**

```
FileInputStream fis = new FileInputStream("a.txt");
System.out.println(fis.read());
```

A. Byte value
B. Character
C. String
D. Error
**Answer:** A

---

**14.**

```
BufferedInputStream bis = new BufferedInputStream(new FileInputStream("a.txt"));
System.out.println(bis.read());
```

A. Faster read
B. Slower
C. Error
D. Same output as FileInputStream
**Answer:** D

---

**15.**

```
RandomAccessFile raf = new RandomAccessFile("a.txt","rw");
raf.seek(2);
raf.write('X');
```

A. Writes at start
B. Writes at position 2
C. Error
D. End
**Answer:** B

---

## ☕ Java 8 Coding MCQs

---

**16.**

```
List<Integer> list = Arrays.asList(1,2,3);
list.forEach(System.out::print);
```

A. 123
B. 1 2 3
C. Error
D. Null
**Answer:** A

---

**17.**

```
List<Integer> list = Arrays.asList(1,2,3,4);
list.stream().filter(x -> x%2==0).forEach(System.out::print);
```

A. 1234
B. 24
C. 13
D. Error
**Answer:** B

---

**18.**

```
List<Integer> list = Arrays.asList(1,2,3);
list.stream().map(x -> x*2).forEach(System.out::print);
```

A. 123
B. 246
C. 6
D. Error
**Answer:** B

---

**19.**

```
Optional<String> opt = Optional.of("Hi");
System.out.println(opt.get());
```
A. Hi
B. Error
C. Null
D. Optional
**Answer:** A

---

**20.**
```
Optional<String> opt = Optional.empty();
System.out.println(opt.orElse("Default"));
```
A. Error
B. Default
C. Null
D. Optional
**Answer:** B

---

**21.**
```
List<String> list = Arrays.asList("a","b","c");
System.out.println(list.stream().findFirst().get());
```
A. a
B. b
C. c
D. Error
**Answer:** A

---

**22.**
```
System.out.println(Stream.of(1,2,3).reduce(0, (a,b)->a+b));
```
A. 6
B. 123
C. 0
D. Error
**Answer:** A

---

**23.**
```
List<List<Integer>> list = Arrays.asList(Arrays.asList(1,2), Arrays.asList(3,4));
list.stream().flatMap(x -> x.stream()).forEach(System.out::print);
```
A. 1234
B. [1,2,3,4]
C. Error
D. Nested
**Answer:** A

---

**24.**
```
Predicate<Integer> p = x -> x > 5;
System.out.println(p.test(10));
```
A. true
B. false

C. Error
D. Null
**Answer:** A

---

**25.**
```
Function<Integer,Integer> f = x -> x*x;
System.out.println(f.apply(4));
```
A. 8
B. 16
C. 4
D. Error
**Answer:** B

---

**26.**
```
Consumer<String> c = x -> System.out.print(x);
c.accept("Hi");
```
A. Hi
B. Error
C. Null
D. Nothing
**Answer:** A

---

**27.**
```
Supplier<Double> s = () -> Math.random();
System.out.println(s.get());
```
A. Random number
B. Error
C. 0
D. 1
**Answer:** A

---

**28.**
```
List<Integer> list = Arrays.asList(5,3,1);
list.stream().sorted().forEach(System.out::print);
```
A. 531
B. 135
C. 513
D. Error
**Answer:** B

---

**29.**
```
System.out.println(Stream.of("A","B","C").count());
```
A. 3
B. ABC
C. Error
D. 0
**Answer:** A

---

**30.**

```
List<String> list = Arrays.asList("a","bb","ccc");
System.out.println(list.stream().map(String::length).reduce(0, Integer::sum));
```

A. 6
B. 3
C. Error
D. 0
**Answer:** A

## Quantitative Aptitude Questions

**1. Find the missing number:**
2, 6, 12, 20, ?
A. 28
B. 30
C. 32
D. 34
**Answer:** B
**Explanation:** Pattern: +4, +6, +8, +10 → 20 + 10 = 30

---

**2.**
3, 9, 27, 81, ?
A. 162
B. 243
C. 324
D. 200
**Answer:** B
**Explanation:** Multiply by 3 → 81 × 3 = 243

---

**3.**
5, 10, 20, 40, ?
A. 60
B. 70
C. 80
D. 100
**Answer:** C
**Explanation:** Multiply by 2

---

**4.**
1, 4, 9, 16, ?
A. 20
B. 24
C. 25
D. 36
**Answer:** C
**Explanation:** Squares → $5^2 = 25$

---

**5.**

7, 14, 28, 56, ?
A. 84
B. 100
C. 112
D. 120
**Answer:** C
**Explanation:** Multiply by 2

---

**6.**
2, 5, 10, 17, ?
A. 24
B. 26
C. 30
D. 32
**Answer:** B
**Explanation:** Pattern: +3, +5, +7, +9

---

**7.**
11, 13, 17, 19, ?
A. 21
B. 23
C. 25
D. 27
**Answer:** B
**Explanation:** Prime numbers

---

**8.**
4, 6, 9, 13, ?
A. 16
B. 17
C. 18
D. 20
**Answer:** C
**Explanation:** +2, +3, +4, +5

---

**9.**
100, 90, 80, 70, ?
A. 60
B. 65
C. 75
D. 50
**Answer:** A
**Explanation:** -10 pattern

---

**10.**
1, 1, 2, 3, 5, ?
A. 6
B. 7

C. 8
D. 9
**Answer:** C
**Explanation:** Fibonacci series

---

**Wrong Number Detection**

---

**11.**
2, 4, 8, 16, 31, 64
A. 8
B. 16
C. 31
D. 64
**Answer:** C
**Explanation:** Should be 32 (×2 pattern)

---

**12.**
3, 6, 12, 24, 50
A. 6
B. 12
C. 24
D. 50
**Answer:** D
**Explanation:** Should be 48

---

**13.**
1, 4, 9, 15, 25
A. 4
B. 9
C. 15
D. 25
**Answer:** C
**Explanation:** Should be 16 (square series)

---

**14.**
5, 10, 20, 45, 80
A. 10
B. 20
C. 45
D. 80
**Answer:** C
**Explanation:** Pattern ×2 → 40 expected

---

**15.**
2, 3, 5, 7, 11, 14
A. 5
B. 7

C. 11
D. 14
**Answer:** D
**Explanation:** Not a prime

---

**16.**
10, 20, 30, 50, 60
A. 20
B. 30
C. 50
D. 60
**Answer:** C
**Explanation:** Pattern +10 broken

---

**17.**
4, 8, 16, 33, 64
A. 8
B. 16
C. 33
D. 64
**Answer:** C
**Explanation:** Should be 32

---

**18.**
1, 8, 27, 65, 125
A. 8
B. 27
C. 65
D. 125
**Answer:** C
**Explanation:** Should be 64 (cubes)

---

**19.**
6, 12, 24, 48, 100
A. 24
B. 48
C. 100
D. 12
**Answer:** C
**Explanation:** Should be 96

---

**20.**
9, 18, 36, 70, 144
A. 18
B. 36
C. 70
D. 144

**Answer:** C
**Explanation:** Should be 72

---

## 🔤 Letter / Number Patterns

---

**21.**
A, C, F, J, ?
A. M
B. N
C. O
D. P
**Answer:** C
**Explanation:** +2, +3, +4, +5 → O

---

**22.**
Z, X, U, Q, ?
A. M
B. N
C. L
D. K
**Answer:** C
**Explanation:** -2, -3, -4, -5

---

**23.**
B, E, H, K, ?
A. M
B. N
C. O
D. P
**Answer:** B
**Explanation:** +3 pattern

---

**24.**
AA, BB, CC, ?
A. DD
B. EE
C. FF
D. GG
**Answer:** A

---

**25.**
AZ, BY, CX, ?
A. DW
B. DV
C. EW
D. EV
**Answer:** A
**Explanation:** A→Z, B→Y pattern

---

**26.**
1A, 2C, 3E, 4G, ?
A. 5H
B. 5I
C. 5J
D. 5K
**Answer:** B
**Explanation:** Letters +2 → I

---

**27.**
AB, DE, GH, ?
A. IJ
B. JK
C. KL
D. LM
**Answer:** B
**Explanation:** +3 pattern

---

**28.**
M, O, R, V, ?
A. Y
B. Z
C. X
D. W
**Answer:** B
**Explanation:** +2, +3, +4, +5

---

**29.**
A1, C3, E5, ?
A. G7
B. G8
C. H7
D. F7
**Answer:** A
**Explanation:** +2 pattern

---

**30.**
2B, 4D, 6F, ?
A. 8H
B. 8G
C. 10H
D. 10G
**Answer:** A
**Explanation:** +2 numbers & letters

**Coding Question**

---

**1. Count Words in File**
**Description:**
You need to read a file and count how many words it contains. A word is usually separated by spaces.
**Example:**
File content:
Java is easy to learn
Words → Java, is, easy, to, learn
**Output:** 5

---

**2. Count Vowels in File**
**Description:**
Read file content and count vowels (a, e, i, o, u), ignoring case.
**Example:**
Hello Java
Vowels → e, o, a, a
**Output:** 4

---

**3. Find Longest Word**
**Description:**
Read all words and identify the word with maximum length.
**Example:**
Java programming language
Longest word → programming
**Output:** programming

---

**4. Convert File Content to Uppercase**
**Description:**
Read text from one file and write the same content in uppercase to another file.
**Example:**
Input → hello world
Output → HELLO WORLD

---

**5. Remove Extra Spaces**
**Description:**
Remove multiple spaces and keep only a single space between words.
**Example:**
Input → Java is powerful
Output → Java is powerful

---

**6. Reverse Each Line**
**Description:**
Each line in the file should be reversed individually.
**Example:**
Input:

Hello
Java
Output:
olleH
avaJ

---

**7. Count Lines, Words, Characters**
**Description:**
You need to count:

- Total lines
- Total words
- Total characters

**Example:**
Input:
Hello Java
Good Day
Lines → 2
Words → 4
Characters → 17 (including spaces)

---

**8. Replace Word in File**
**Description:**
Replace all occurrences of a specific word with another.
**Example:**
Input → Java is good. Java is fast.
Replace Java → Python
Output → Python is good. Python is fast.

---

**9. Find Duplicate Words**
**Description:**
Identify words that appear more than once.
**Example:**
Java is easy and Java is powerful
Duplicate words → Java, is
**Output:** Java, is

---

**10. Sort Lines Alphabetically**
**Description:**
Sort all lines in a file in alphabetical order.
**Example:**
Input:
Banana
Apple
Mango
Output:
Apple
Banana
Mango

## ☕ Java 8 Stream API (Number Processing)

---

**1. Find Even Numbers**
**Description:**
Filter numbers divisible by 2.
**Example:**
Input → [1, 2, 3, 4, 5]
Output → [2, 4]

---

**2. Find Odd Numbers**
**Description:**
Filter numbers not divisible by 2.
**Example:**
Input → [1, 2, 3, 4]
Output → [1, 3]

---

**3. Square Each Number**
**Description:**
Transform each number into its square.
**Example:**
Input → [2, 3, 4]
Output → [4, 9, 16]

---

**4. Sum of All Numbers**
**Description:**
Add all numbers in a list.
**Example:**
Input → [1, 2, 3, 4]
Output → 10

---

**5. Find Maximum Number**
**Description:**
Find the largest number.
**Example:**
Input → [3, 7, 2, 9]
Output → 9

---

**6. Find Minimum Number**
**Description:**
Find the smallest number.
**Example:**
Input → [3, 7, 2, 9]
Output → 2

---

**7. Count Numbers Greater Than 10**
**Description:**
Count how many numbers are greater than 10.

**Example:**
Input → [5, 12, 18, 3]
Output → 2

---

**8. Remove Duplicates**
**Description:**
Remove repeated elements.
**Example:**
Input → [1, 2, 2, 3, 3]
Output → [1, 2, 3]

---

**9. Sort Numbers**
**Description:**
Arrange numbers in ascending order.
**Example:**
Input → [5, 2, 8]
Output → [2, 5, 8]

---

**10. Sort in Descending Order**
**Description:**
Arrange numbers from largest to smallest.
**Example:**
Input → [5, 2, 8]
Output → [8, 5, 2]

---

**11. Find First Element**
**Description:**
Return the first element of a list.
**Example:**
Input → [10, 20, 30]
Output → 10

---

**12. Find Any Element**
**Description:**
Return any element (useful in parallel processing).
**Example:**
Input → [10, 20, 30]
Output → 10 or 20 or 30

---

**13. Check All Match Condition**
**Description:**
Check if all elements satisfy a condition.
**Example:**
Input → [2, 4, 6]
Condition → even
Output → true

---

**14. Check Any Match**

**Description:**
Check if at least one element satisfies condition.
**Example:**
Input → [1, 3, 6]
Condition → even
Output → true

---

**15. Convert List to Set**
**Description:**
Convert list into set to remove duplicates.
**Example:**
Input → [1, 2, 2, 3]
Output → [1, 2, 3]

---

**16. Multiply All Numbers**
**Description:**
Find product of all elements.
**Example:**
Input → [2, 3, 4]
Output → 24

---

**17. Find Second Highest Number**
**Description:**
Find second largest element.
**Example:**
Input → [10, 20, 30, 40]
Output → 30

---

**18. Group Even and Odd Numbers**
**Description:**
Separate numbers into even and odd groups.
**Example:**
Input → [1, 2, 3, 4]
Output →
Even: [2, 4]
Odd: [1, 3]

---

**19. Find Frequency of Each Number**
**Description:**
Count how many times each number appears.
**Example:**
Input → [1, 2, 2, 3, 3, 3]
Output →
1 → 1
2 → 2
3 → 3

---

**20. Sum of Squares of Even Numbers**

**Description:**
Take even numbers → square them → sum.
**Example:**
Input → [1, 2, 3, 4]
Even → [2, 4]
Squares → [4, 16]
Output → 20